

Musterklausur Programmierung II

Daniel Appenmaier, 27. Juli 2026

Aufgabe	Maximale Punktzahl	Erreichte Punktzahl
1	17	
2	16	
3	19	
Gesamt	52	

Hinweise

- Die in dieser Klausur verwendeten Personenbezeichnungen beziehen sich – sofern nicht anders kenntlich gemacht – auf alle Geschlechter
- Pakete und Klassenimporte müssen nicht angegeben werden
- Die Klassen und Annotationen der Bibliotheken *Lombok*, *JUnit 5* und *Mockito* dürfen verwendet werden
- Es kann davon ausgegangen werden, dass sämtliche Klassen entsprechende Implementierungen der Object-Methoden besitzen
- So nicht anders angegeben sollen Konstruktoren, Setter, Getter sowie die Object-Methoden wie in der Vorlesung gezeigt implementiert werden
- Die Konsolenausgaben-Methoden der Klasse `PrintStream` dürfen sinnvoll gekürzt geschrieben werden (zum Beispiel `sysout(„Hello World“)` statt `System.out.println(„Hello World“)`)
- Methoden- und Attributsbezeichner dürfen sinnvoll gekürzt geschrieben werden (zum Beispiel `getLWMCP()` statt `getLectureWithMostCreditPoints()`)

Wörterbuch

Englisch	Deutsch	Englisch	Deutsch
description	Beschreibung	robot	Roboter
fabrication date	Fabrikationsdatum	serial number	Seriennummer
fabrication year	Fabrikationsjahr	series	Serie
factory	Fabrik	terminator	Terminator
humanoid	menschenähnlich	weight	Gewicht

Aufgabe 1 (17 Punkte)

Erstelle die Klassen `Terminator` (7 Punkte) und `RobotFactory` (10 Punkte) anhand des abgebildeten Klassendiagramms.

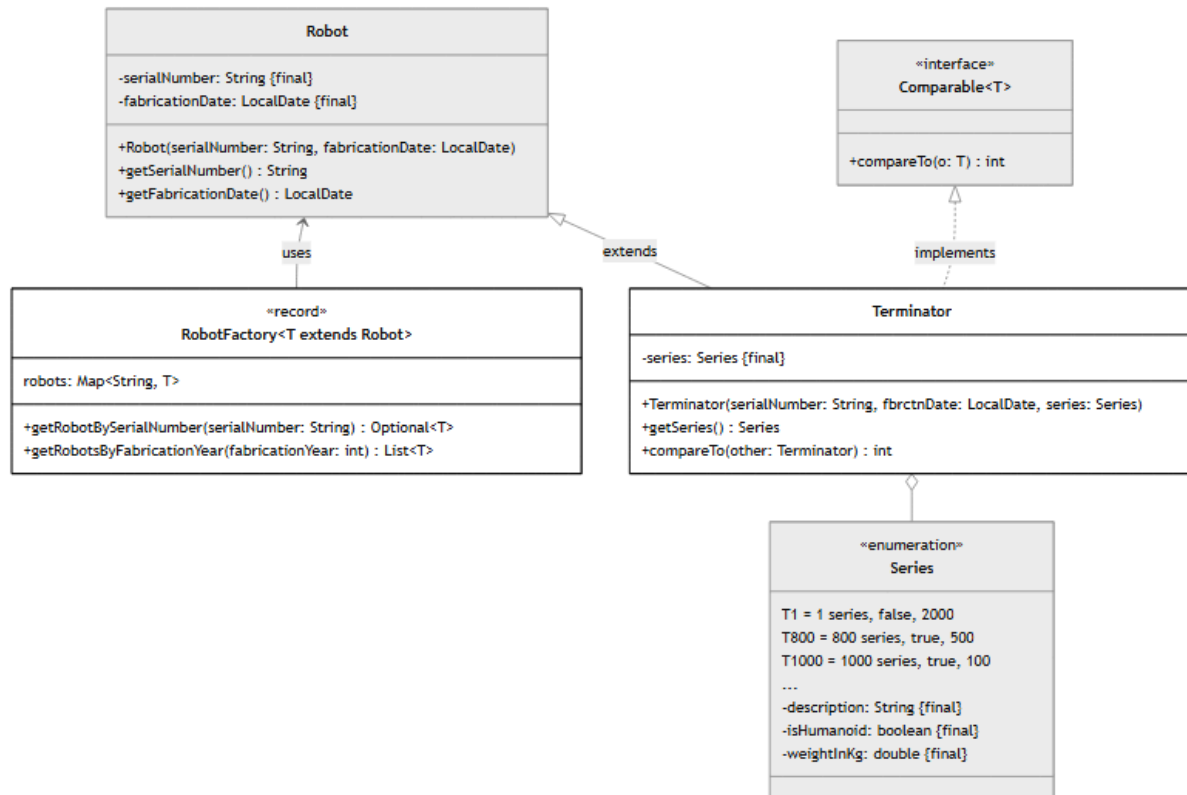


Abbildung 1: Klassendiagramm Aufgabe 1

Hinweise zur Klasse *Terminator*

Die Methode `int compareTo(other: Terminator)` soll so implementiert werden, dass Terminatoren absteigend nach ihrem Gewicht sortiert werden können (2 Punkte).

Hinweise zur Klasse *RobotFactory*

- Die Schlüssel-Werte-Paare des Assoziativspeichers `robots` beinhalten als Schlüssel die Seriennummer sowie als Wert den dazugehörigen Roboter
- Die Methode `Optional<T> getRobotBySerialNumber(serialNumber: String)` soll den Roboter zur eingehenden Seriennummer zurückgeben (4 Punkte)
- Die Methode `List<T> getRobotsByFabricationYear(fabricationYear: int)` soll alle Roboter zum eingehenden Fabrikationsjahr zurückgeben (4,5 Punkte)

Aufgabe 2 (16 Punkte)

Erstelle die JUnit-5-Testklasse `RobotFactoryTest` anhand des abgebildeten Klassendiagramms.

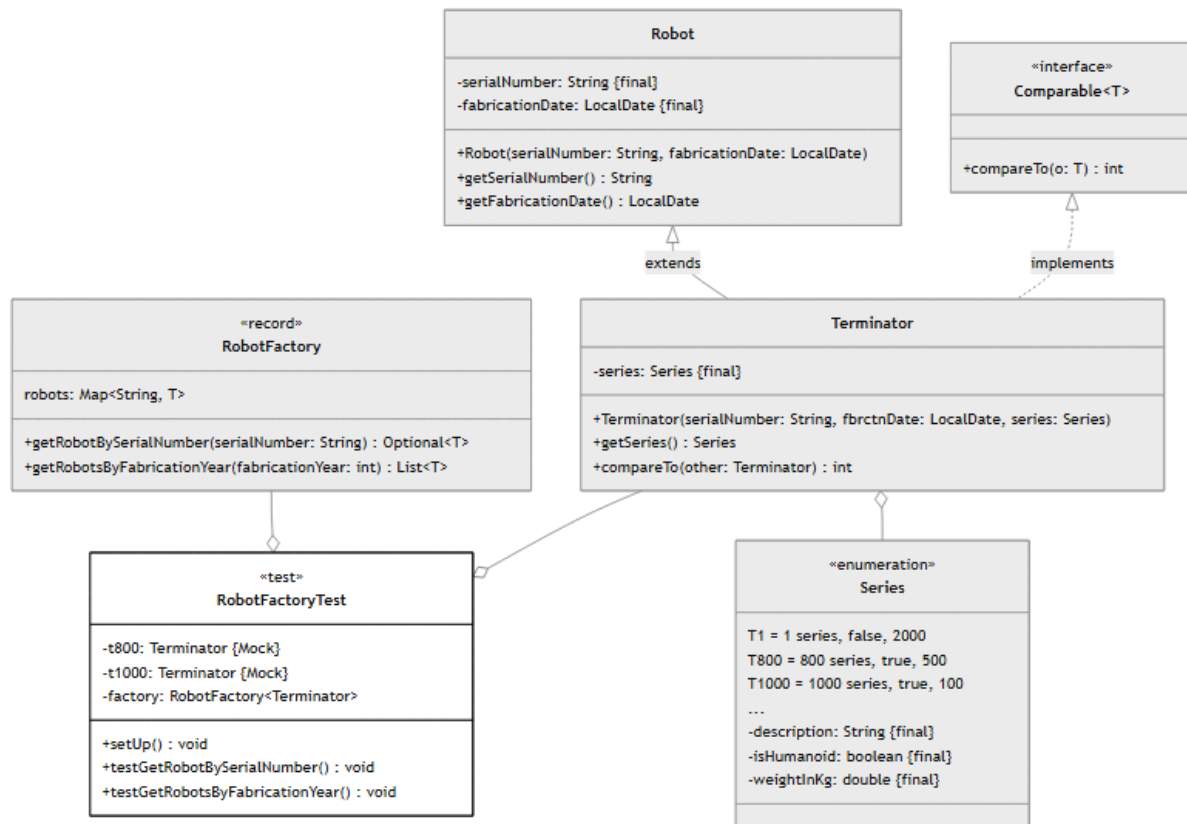


Abbildung 2: Klassendiagramm Aufgabe 2

Hinweise zur Klasse `RobotFactoryTest`

- Die Lebenszyklus-Methode `void setUp()` soll alle Mock-Objekte initialisieren, eine Roboterfabrik erstellen, diese Roboterfabrik dem entsprechenden Attribut zuweisen und in der Roboterfabrik für den Schlüssel 800-1 als Wert den Terminator T800 sowie für den Schlüssel 1000-1 als Wert den Terminator T1000 setzen (4,5 Punkte)
- Die Testmethode `void testGetRobotBySerialNumber()` soll prüfen, ob beim Aufruf der Methode `Optional<T> getRobotBySerialNumber(serialNumber: String)` auf die Roboterfabrik mit der Seriennummer 1000-0 ein leeres `Optional` und beim Aufruf mit der Seriennummer 1000-1 der Terminator T1000 als `Optional` zurückgegeben wird (4 Punkte)
- Die Testmethode `void testGetRobotsByFabricationYear()` soll prüfen, ob beim Aufruf der Methode `List<T> getRobotsByFabricationYear(fabricationYear: int)` auf die Roboterfabrik mit dem Wert 2029 eine Liste mit dem Terminator T800 als Eintrag zurückgegeben wird. Hierzu soll das Mock-Objekt T800 beim Aufruf der Methode `LocalDate getFabricationDate()` das Datum 24.03.2029 und das Mock-Objekt T1000 das Datum 12.07.2032 zurückgeben (4,5 Punkte)

Aufgabe 3 (19 Punkte)

Erstelle die Klasse `TerminatorQueries` anhand des abgebildeten Klassendiagramms.

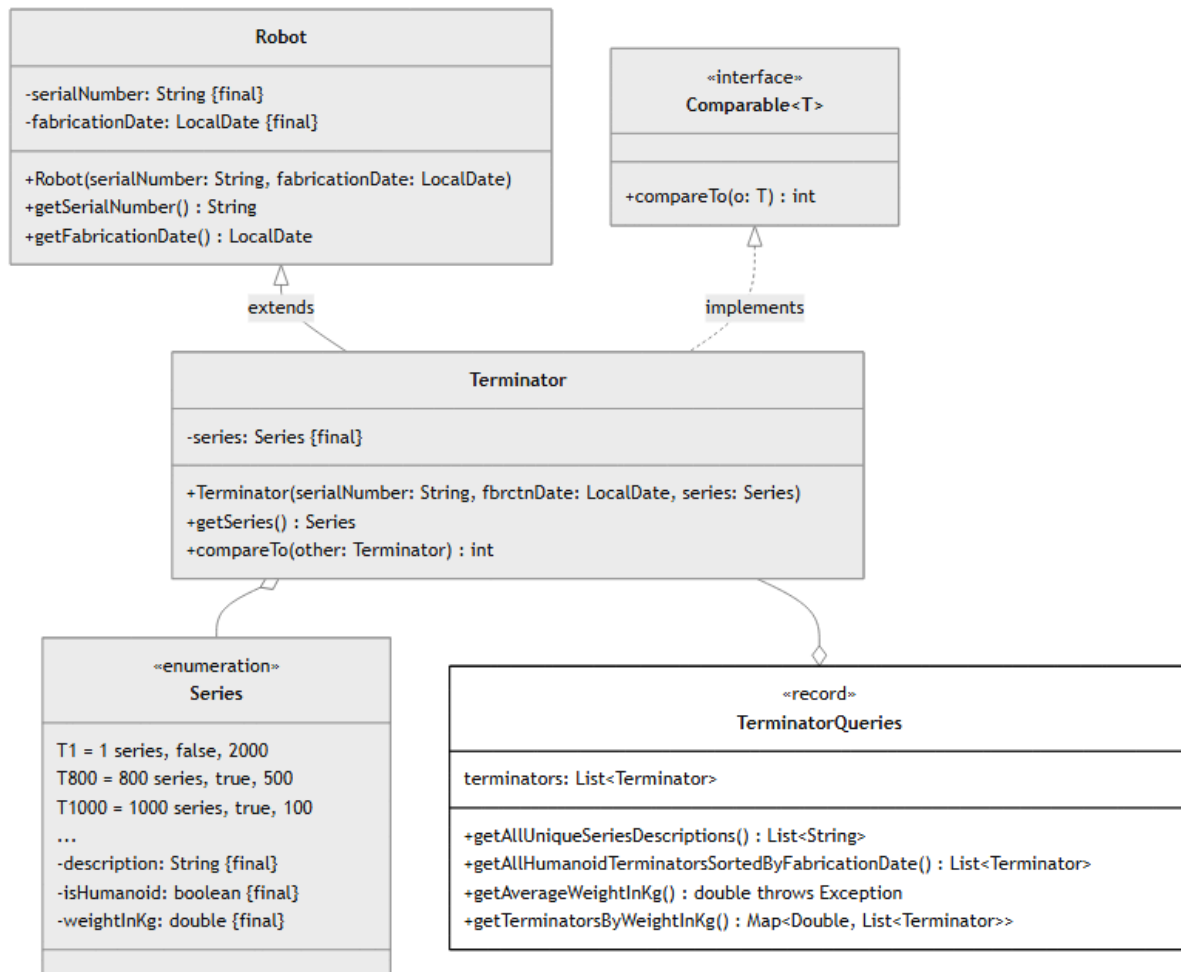


Abbildung 3: Klassendiagramm Aufgabe 3

Hinweise zur Klasse *TerminatorQueries*

- Die Methode `List<String> getAllUniqueSeriesDescriptions()` soll die Serienbeschreibungen aller Terminatoren ohne Dopplungen zurückgeben (3,5 Punkte)
- Die Methode `List<Terminator> getAllHumanoidTerminatorsSortedByFabricationDate()` soll alle menschenähnlichen Terminatoren absteigend sortiert nach ihrem Fabrikationsdatum zurückgeben (5 Punkte)
- Die Methode `double getAverageWeightInKg() throws Exception` soll das durchschnittliche Gewicht aller Terminatoren zurückgeben. Für den Fall, dass kein durchschnittliches Gewicht berechnet werden kann (leere Liste), soll die Ausnahme `Exception` ausgelöst werden (6 Punkte)
- Die Methode `Map<Double, List<Terminator>> getTerminatorsByWeightInKg()` soll alle Terminatoren gruppiert nach ihrem Gewicht zurückgeben (3,5 Punkte)

Cheatsheet

java.lang

```
Boolean - int compare(x: boolean, y: boolean) //static
Boolean - Boolean valueOf(s: String) //static
Boolean - Boolean valueOf(b: boolean) //static
Comparable<T> - int compareTo(o: T)
Double - int compare(d1: double, d2: double) //static
Double - Double valueOf(s: String) //static
Double - Double valueOf(d: double) //static
Integer - int compare(x: int, y: int) //static
Integer - Integer valueOf(s: String) //static
Integer - Integer valueOf(i: int) //static
Long - long compare(x: long, y: long) //static
Long - Long valueOf(s: String) //static
Long - Long valueOf(l: long) //static
Object - boolean equals(object: Object)
Object - int hashCode()
Object - String toString()
Runnable - void run()
String - char charAt(index: int)
String - int length()
String - String[] split(regex: String)
String - String toLowerCase()
String - String toUpperCase()
System - long currentTimeMillis() //static
```

java.time

```
LocalDate - int getDayOfMonth()
LocalDate - int getDayOfYear()
LocalDate - Month getMonth()
LocalDate - int getMonthValue()
LocalDate - int getYear()
LocalDate - LocalDate now() //static
LocalDate - LocalDate of(year: int, month: int, dayOfMonth: int) //static
LocalDate - LocalDate parse(text: CharSequence) //static
LocalTime - int getHour()
LocalTime - int getMinute()
LocalTime - int getSecond()
LocalTime - LocalTime now() //static
LocalTime - LocalTime of(hour: int, minute: int, second: int) //static
LocalTime - LocalTime parse(text: CharSequence) //static
```

java.io

```
PrintStream - void print(obj: Object)
PrintStream - PrintStream printf(String format, Object... args)
PrintStream - void println()
PrintStream - void println(x: Object)
```

java.util

```
Collections - void sort(list: List<T>) //static
Collections - void sort(list: List<T>, c: Comparator<T>) //static
Comparator<T> - int compare(o1: T, o2: T)
Comparator<T> - Comparator<T> comparing(keyExtractor: Function<T,U>) //static
Entry<K,V> - K getKey()
Entry<K,V> - V getValue()
Entry<K,V> - V setValue(value: V)
Enumeration - Enumeration valueOf(arg0: String) //static
Enumeration - Enumeration[] values() //static
List<E> - boolean add(e: E)
List<E> - void add(index: int, element: E)
List<E> - boolean contains(o: Object)
List<E> - void forEach(action: Consumer<T>)
List<E> - E get(index: int)
List<E> - List<E> of(elements: E...) //static
List<E> - E remove(index: int)
List<E> - boolean remove(o: Object)
List<E> - List<T> reversed()
List<E> - int size()
List<E> - void sort(c: Comparator<T>)
Map<K,V> - boolean containsKey(key: Object)
Map<K,V> - boolean containsValue(value: Object)
Map<K,V> - Set<Entry<K,V>> entrySet()
Map<K,V> - void forEach(action: BiConsumer<K,V>)
Map<K,V> - V get(key: Object)
Map<K,V> - Set<K> keySet()
Map<K,V> - V put(key: K, value: V)
Map<K,V> - V putIfAbsent(key: K, value: V)
Map<K,V> - Collection<V> values()
Optional<T> - Optional<T> empty() //static
Optional<T> - T get()
Optional<T> - void ifPresent(action: Consumer<T>)
Optional<T> - void ifPresentOrElse(action: Consumer<T>, emptyAction: Runnable)
Optional<T> - boolean isPresent()
Optional<T> - Optional<T> of(t: T) //static
Optional<T> - Optional<T> ofNullable(t: T) //static
Optional<T> - T orElse(other: T)
OptionalDouble - OptionalDouble empty() //static
OptionalDouble - double getAsDouble()
OptionalDouble - void ifPresent(action: DoubleConsumer)
OptionalDouble - void ifPresentOrElse(action: DoubleConsumer, emptyAction: Runnable)
OptionalDouble - boolean isPresent()
OptionalDouble - OptionalDouble of(value: double) //static
OptionalDouble - double orElse(other: double)
Random - int nextInt(origin: int, bound: int)
```

java.util.function

```
BiConsumer - void accept(t: T, u: U)
Consumer<T> - void accept(t: T)
DoubleConsumer - void accept(value: double)
Function<T,R> - R apply(t: T)
Predicate<T> - boolean test(t: T)
ToDoubleFunction<T,R> - double applyAsDouble(value: T)
ToIntFunction<T,R> - int applyAsInt(value: T)
```

java.util.stream

```
Collectors - Collector<T,?,Map<K,List<T>>> groupingBy(classifier: Function<T,K>) //static
Collectors - Collector<CharSequence,?,String> joining(delimiter: CharSequence) //static
Collectors - Collector<T,?,Map<Boolean,List<T>>> partitioningBy(predicate:
Predicate<T>) //static
Collectors - Collector<T,?,Map<K,U>> toMap(keyMapper: Function<T,K>, valueMapper:
Function<T,U>) //static
Collectors - Collector<T,?,Set<T>> toSet() //static
DoubleStream - OptionalDouble average()
DoubleStream - double sum()
IntStream - OptionalDouble average()
IntStream - int sum()
Stream<T> - boolean allMatch(predicate: Predicate<T>)
Stream<T> - boolean anyMatch(predicate: Predicate<T>)
Stream<T> - R collect(collector: Collector<T,A,R>)
Stream<T> - long count()
Stream<T> - Stream<T> distinct()
Stream<T> - Stream<T> filter(predicate: Predicate<T>)
Stream<T> - Optional<T> findAny()
Stream<T> - Optional<T> findFirst()
Stream<T> - void forEach(action: Consumer<T>)
Stream<T> - Stream<T> limit(maxSize: long)
Stream<T> - Stream<R> map(mapper: Function<T,R>)
Stream<T> - DoubleStream mapToDouble(mapper: ToDoubleFunction<T,R>)
Stream<T> - IntStream mapToInt(mapper: ToIntFunction<T,R>)
Stream<T> - Optional<T> max(comparator: Comparator<T>)
Stream<T> - Optional<T> min(comparator: Comparator<T>)
Stream<T> - boolean noneMatch(predicate: Predicate<T>)
Stream<T> - Stream<T> skip(n: long)
Stream<T> - Stream<T> sorted()
Stream<T> - Stream<T> sorted(comparator: Comparator<T>)
Stream<T> - List<T> toList()
```


org.junit.jupiter.api

```
Assertions - void assertEquals(expected: Object, actual: Object) //static
Assertions - void assertFalse(condition: boolean) //static
Assertions - void assertNotEquals(expected: Object, actual: Object) //static
Assertions - void assertNotNull(actual: Object) //static
Assertions - void assertNotSame(expected: Object, actual: Object) //static
Assertions - void assertNull(actual: Object) //static
Assertions - void assertSame(expected: Object, actual: Object) //static
Assertions - T assertThrows(expectedType: Class<T>, executable: Executable) //static
Assertions - void assertTrue(condition: boolean) //static
Executable - void execute()
```

org.mockito / org.mockito.stubbing

```
Mockito - OngoingStubbing<T> when(methodCall: T) //static
MockitoAnnotations - AutoCloseable openMocks(testClass: Object) //static
OngoingStubbing<T> - OngoingStubbing<T> thenReturn(value: T)
```